Publish–Subscribe Messaging System over TCP with SDN

Comprehensive Project Architecture, Role Allocation & Execution Guide for Team Members

Course: UE25CS243A — Computer Networks	**Track:** Project #9 (Socket + SDN)
Total Marks: 40 (D1: 15M | D2: 25M)	**Team Size:** 3 Members
GitHub Repo: github.com/chandrugworkspace/sdn-tcp-pubsub	**Architecture:** 4-Switch Diamond Multi-Path Topology

1. Project Overview & Architecture

This project implements an intelligent, reliable publish–subscribe messaging infrastructure combining **raw TCP socket programming** with an **OpenFlow 1.3 Software-Defined Networking (SDN) controller**. The centralized Broker handles multi-threaded client connections, topic subscriptions, and ACKs, while the SDN controller dynamically classifies topic traffic at wire speed and steers packets along distinct physical paths.

Path Component	Switch Route	Bandwidth & Latency	Traffic Assignment
Upper Fast Path	s1 ➔ s2 ➔ s4	**20 Mbps** | 2 ms / hop (~8 ms end-to-end)	Port 5001 (Priority topics) & Dynamically Promoted bursts
Lower Standard Path	s1 ➔ s3 ➔ s4	**5 Mbps** | 25 ms / hop (~58 ms end-to-end)	Port 5002 (Standard topics: sensors/temp)
Automatic Failover	s1 ➔ s3 ➔ s4 (Backup)	Sub-second redirect via Priority 60 OpenFlow rules	Active whenever link s1–s2 goes DOWN (Zero dropped conns)

2. Team Member Ownership & Rubric Mapping (40 Marks)

Each team member owns specific codebase modules and covers dedicated components of the lab evaluation rubric:

Member 1	**TCP Socket & Pub–Sub Lead**	`pubsub/protocol.py`
		`pubsub/broker.py`
		`pubsub/subscriber.py`
		`pubsub/publisher.py`
			• TCP/UDP Socket Implementation (4M)
			• Complete Pub-Sub Application (4M)
			• Framing, Threading & Reliability ACKs

Member 2	**Mininet & SDN Controller Lead**	`topology/pubsub_topo.py` `controller/sdn_controller.py`	• Mininet Virtual Network Topology (4M) • OpenFlow 1.3 Controller Steering (6M) • Loop-Free Directed ARP

Member 3	**Integration, Testing & Benchmarking Lead**	`benchmarks/plot_metrics.py`
		`run_local_demo.py`
		`Webhook Integration logic`
		`README.md`
			• Problem Understanding & Architecture (2M)
			• Socket–SDN Webhook Integration (3M)
			• Robustness & Link Failover Demo (4M)
			• Quantitative Evaluation & Graphs (5M)
			• Documentation & Viva (6M)

3. Environment Setup by Operating System

All teammates should clone the shared GitHub repository first:
git clone https://github.com/chandrugworkspace/sdn-tcp-pubsub.git && cd sdn-tcp-pubsub

OS Environment	Setup Steps & Commands

CachyOS / Arch Linux *(Native Execution)*	`sudo pacman -Syu mininet openvswitch iproute2 iperf3 uv`
	`sudo systemctl enable --now ovs-vswitchd`
	`uv venv --python 3.11 .venv && source .venv/bin/activate.fish`
	`VIRTUAL_ENV=.venv uv pip install -r requirements.txt`

Windows 10/11 *(Ubuntu 22.04 LTS VM)*	*`*Note: Use VirtualBox or VMware (standard WSL2 lacks OVS kernel modules)*`*
	`sudo apt update && sudo apt install mininet openvswitch-switch iproute2 git python3-venv -y`
	`python3 -m venv .venv && source .venv/bin/activate`
	`pip install -r requirements.txt`

macOS (Apple / Intel) *(UTM / VMware Fusion)*	Install an Ubuntu 22.04 LTS VM using UTM (free). Follow the exact same Ubuntu steps above.

4. Step-by-Step Execution Guide by Role

A. Member 1: Standalone Socket Pub-Sub Testing (Deliverable 1)

Member 1 can test the entire Pub-Sub system without Mininet or root permissions across separate terminal tabs or with the automated runner:

Fast 5-Second Automated Test:
```
python3 run_local_demo.py
```

Interactive 4-Terminal Workflow:
- **Terminal 1 (Broker):** `python3 pubsub/broker.py --priority-port 5001 --std-port 5002`
- **Terminal 2 (Subscriber 1):** `python3 pubsub/subscriber.py --port 5002 --topics sensors/temp --csv std.csv`
- **Terminal 3 (Subscriber 2):** `python3 pubsub/subscriber.py --port 5001 --topics alerts/fire --csv prio.csv`
- **Terminal 4 (Publisher):** `python3 pubsub/publisher.py --port 5002 --topic sensors/temp --count 20`

B. Members 2 & 3: Mininet + SDN Controller Integration (Deliverable 2)

Demonstrates wire-speed flow classification, dynamic threshold promotion, and link-down failover:

1. Start Controller (Terminal 1):
```
.venv/bin/osken-manager --ofp-tcp-listen-port 6633 controller/sdn_controller.py
```
(Starts OpenFlow engine on port 6633 and built-in REST webhook API on port 8080)

2. Launch Mininet Diamond Topology (Terminal 2):
```
sudo mn -c && sudo python3 topology/pubsub_topo.py
```

3. Inside Mininet CLI:
```
mininet> pingall
mininet> h3 python3 pubsub/broker.py --sdn-url http://127.0.0.1:8080 > broker.log 2>&1 &
mininet> h4 python3 pubsub/subscriber.py --broker 10.0.0.3 --port 5002 --topics sensors/temp --csv
std_latency.csv > sub.log 2>&1 &
mininet> h1 python3 pubsub/publisher.py --broker 10.0.0.3 --port 5002 --topic sensors/temp --count 25
--interval 0.3
```

4. Observe Dynamic Promotion (Message #15):
```
mininet> sh tail -n 25 sub.log ➔ Latency drops from ~58 ms down to ~8 ms!
```

5. Demonstrate Link Failure & Fast Failover:
```
mininet> link s1 s2 down ➔ Controller instantly switches traffic over s3 without dropping connection!
mininet> link s1 s2 up ➔ Normal priority paths restored.
```

6. Generate Benchmark Comparison Graph:
```
mininet> exit
python3 benchmarks/plot_metrics.py std_latency.csv prio_latency.csv latency_comparison.png
```

5. Viva Examination Preparation & Technical Q&A;

Use these questions and model answers during Deliverable 1 and Deliverable 2 evaluations:

[Member 1 (Socket Lead)] Q: Why use a 4-byte length prefix (!I) instead of string delimiters like newline (\n)?

Answer: TCP is a continuous byte-stream protocol without built-in message boundaries. A single recv(1024) call can contain half a message or three concatenated messages. The 4-byte unsigned integer prefix gives the exact byte length, allowing recv_exact() to reconstruct the JSON object without fragmentation or corruption.

[Member 1 (Socket Lead)] Q: How does the Broker prevent concurrency race conditions when multiple clients connect?

Answer: The Broker encapsulates subscription mutations inside a threading.Lock(). During message broadcasting, it takes a snapshot list copy list(self.subscriptions[topic]) so that new subscribers joining or leaving mid-broadcast never trigger dictionary mutation runtime errors.

[Member 2 (SDN Lead)] Q: Why does the diamond topology require a custom controller instead of simple_switch_13?

Answer: The 4 switches form a physical Layer-2 diamond loop (s1-s2-s4-s3-s1). Standard learning switches flood ARP packets, causing an infinite broadcast storm that saturates links. Our controller uses directed, loop-free ARP handling and installs explicit OpenFlow 1.3 flow rules directly.

[Member 2 (SDN Lead)] Q: How are high-priority topics separated from standard topics at wire speed?

Answer: Without performing slow deep packet inspection (DPI) on JSON payloads, the switches inspect Layer-4 destination TCP port numbers in hardware (OFPMatch tcp_dst=5001 vs tcp_dst=5002) and steer packets into egress port 3 (Fast Path) or port 4 (Standard Path).

[Member 3 (Integration Lead)] Q: How does the application layer dynamically influence the SDN network layer?

Answer: The Broker monitors per-topic message volume in real time. When a topic crosses 15 messages, it fires an HTTP POST /sdn/policy webhook to the SDN Controller. The controller dynamically installs priority 50 flow rules on switches s1 and s4, rerouting that host onto the Fast Path mid-stream.

[Member 3 (Integration Lead)] Q: What quantitative performance improvement is achieved by dynamic SDN steering?

Answer: End-to-end delivery latency drops from ~55–60 ms on the 5 Mbps / 25 ms standard path down to ~6–10 ms on the 20 Mbps / 2 ms fast path, achieving an ~85% latency reduction.

6. Collaborative Git Workflow

- **Clone:** git clone https://github.com/chandrugworkspace/sdn-tcp-pubsub.git
- **Update:** git pull origin main before starting any lab session.
- **Branching:** git checkout -b feature/your-name-updates
- **Syncing:** git add . && git commit -m 'your message' && git push origin main